2.4 Constructing and Describing Objects

Constructors make required state available as soon as an object exists. Custom methods then answer useful questions about that state, and `__toString()` supplies a concise string representation.

Initialize a valid object

PHP automatically invokes `__construct()` after `new`:

```
class Person
{
    private string $displayName;
    private string $birthDate;

    public function __construct(string $displayName, string $birthDate)
    {
        $this->displayName = $displayName;
        $this->birthDate = $birthDate;
        // or $this->setDisplayName($displayName);
        // and $this->setBirthDate($birthDate);
        // if not using property hooks
    }

    // Getter and validated setter methods follow.
}

$joe = new Person('Joe');
```

Calling existing setters from the constructor keeps each validation rule in one place. Constructors do not declare a return type.

Constructor property promotion

PHP can declare and assign properties in the constructor signature:

```
class Member
{
    public function __construct(
        private string $displayName,
        private ?string $bio = null,
    ) {
    }
}
```

Promotion is useful when assignment needs no extra logic. An explicit property plus a constructor body is often clearer when validation or transformation is required.

Convert an object to a string

`__toString()` runs when an object is used where PHP expects a string:

```php
public function __toString(): string
{
    return sprintf('%s (%.0f years old)', $this->displayName, $this->age);
}

$joe = new Person('Joe', '1990-01-01');
echo $joe; // Joe (30 years old)
```

The method must be public and return a string. Use it for a short, unsurprising representation—not a large HTML fragment or an operation with side effects.